INCIDENT RESPONSE & DEBUGGING SOP

Equilibrium Bug Triage Playbook

Standard Operating Procedure for Isolating, Diagnosing, and Resolving Issues Rapidly

System: Full-Stack (Frontend, Backend, ML, MongoDB)

Objective: Mean Time to Recovery (MTTR) < 5 Minutes

Target: Zero Crash During Demos & Testing

1. The 5-Step Universal Incident Triage Framework

When an unexpected error or visual glitch appears, follow this systematic diagnostic loop rather than guessing:

Step 1: Isolate the Origin Layer

Determine which tier produced the error:

- **Frontend:** Browser console (F12) shows red unhandled JS exceptions.
- **Network (HTTP):** 4xx or 5xx status codes in the Network tab.
- **Backend:** Terminal 1 logs throw an unhandled promise rejection.
- **ML Microservice:** Terminal 3 shows FastAPI 500 or timeout.

Step 2: Inspect Real Data Payload

Inspect the exact JSON response in the browser Network tab under `Fetch/XHR`. Does the backend return empty arrays `[]` or missing fields? Never guess what data arrived; inspect the raw JSON.

Step 3: Check Terminal Output

Look at the active running terminal. If Node crashed, it will print the exact stack trace with line numbers. If ML crashed, Python traceback will pinpoint the exact feature vector error.

Step 4: Execute Minimal Fix & Verify

Make the surgical code correction. Do a TypeScript compile check (`npx tsc --noEmit`) immediately to catch secondary typing errors before restarting.

2. Component-by-Component Diagnostic Tree

Component	Symptom / Bug	First Diagnostic Action	Resolution Path
FRONTEND UI	White screen of death or dashboard components not rendering.	Open browser DevTools (`Ctrl + Shift + I` or `F12`) → **Console**.	Look for unhandled syntax error (e.g. duplicate variable declaration) or undefined property access (`data?.items`).
FRONTEND STATE	Stale data; completed task still appears or count shows 0.	Inspect **TanStack Query** cache or trigger manual hard-refresh (`Ctrl + Shift + R`).	Verify query keys match in `useQuery` and `queryClient.invalidateQueries({ queryKey: [...] })`.
BACKEND API	HTTP 500 on `/api/schedule/generate` or `/api/tasks`.	Inspect Backend terminal stdout/stderr logs.	Check if MongoDB is running and Mongoose models match schemas. Verify date ranges (`from < to`).
DATABASE	Query returns empty array even though tasks exist in database.	Verify `userId` matching logic in service.	MongoDB stores `userId` as `ObjectId` while JWT passes string. Use `$in: [userId, new ObjectId(userId)]`.
ML SERVICE	Task creation hangs or predicted duration equals null.	Check if FastAPI service on port 8000 is running: `http://localhost:8000/health`.	Fallback to rule-based duration automatically if ML service is unreachable. Verify feature dimension array length (12 features).
REAL-TIME SSE	Schedule does not auto-update when task is extended.	Check Network tab for `/api/schedule/sse` connection status.	Verify SSE token authentication. Ensure client reconnects automatically with exponential backoff on disconnect.

3. Top 6 Equilibrium Specific Edge-Cases & Direct Fixes

Case 1: Tasks Missing on Dashboard but Visible on Schedule Page

Root Cause:

1. **Time Window Mismatch:** Dashboard queried `now` to `endOfDay`, while Schedule page queried the whole 24-hour bracket starting from `00:00:00`. Tasks scheduled earlier in the day were excluded.
2. **Array Slicing:** Slicing only the first 6 items caused early morning blocks (e.g. Sleep at 02:00) to push daytime study tasks off the widget.

```
// Fix: Always start today's query window from midnight:
const from = startOfDay(new Date());
const to = endOfDay(new Date());
// Display up to 8-10 blocks and highlight TASK blocks prominently in Up Next card
```

Case 2: Duplicate Variable Declarations in Component Scope

Root Cause: Redeclaring `const now = new Date()` in the same component body throws a fatal `SyntaxError: Identifier 'now' has already been declared` which causes React to crash silently or fail hot reload.

```
// Verification command to run before testing:
cd frontend ; npx tsc --noEmit
// If TypeScript outputs 0 errors, the component syntax is clean!
```

Case 3: MongoDB ObjectId vs String Type Discrepancy

Root Cause: When tasks are populated from seeds or different endpoints, some documents store `userId` as a Mongoose `ObjectId` and others as a pure string. A strict query `{ userId: req.user.id }` fails to match.

```
// Safe Dual-Matching Pattern used in Equilibrium:
const userObjId = mongoose.Types.ObjectId.isValid(userId)
  ? new mongoose.Types.ObjectId(userId)
  : null;
const filter = userObjId ? { $in: [userId, userObjId] } : userId;
```

Case 4: Timezone Offsets Shifting Sleep & Study Blocks

Root Cause: Using UTC dates on a client machine in local timezone (e.g. IST UTC+5:30) can displace bedtime blocks into the wrong day.

```
// Solution: Equilibrium uses custom timezone utilities:
// `makeZonedDate` and `getZonedParts` in `backend/src/utils/timezone.ts`
// Always pass user's registered timezone or fall back to client local offset.
```

4. 30-Second Live Demo Emergency Playbook (For Presentation Day)

> 🚨 **Emergency Rules If a Bug Happens Live In Front of the Audience:**
>
> 1. **Rule 1: NEVER PANIC OR FREEZE.** Say calmly: *"Notice how our system catches runtime state changes — let me refresh the client cache to show the live synchronization."*
> 2. **Rule 2: The Fast Client Reset (1 Second):** Press `Ctrl + Shift + R` (Hard reload). In 90% of web demo glitches, this clears any stale DOM or local storage cache instantly.
> 3. **Rule 3: If Backend Stopped (5 Seconds):** Keep terminal windows organized side-by-side. If the backend crashed, simply press `Up Arrow → Enter` in the backend terminal to restart `npm run dev`. It restarts in under 3 seconds!
> 4. **Rule 4: If ML Service Times Out (Bypass Feature):** Equilibrium backend has built-in graceful degradation: if Python port 8000 is unreachable, the scheduler automatically falls back to Rule-Based Multipliers. The app does NOT crash!
> 5. **Rule 5: Keep a Pre-Seeded Demo Account:** Have one test account already logged in with 5 completed tasks and a generated schedule. If live task creation acts up, switch tabs to the pre-seeded account immediately!

5. Developer Diagnostic & Healthcheck Command Cheat Sheet

Run these quick terminal one-liners to verify full system health in 10 seconds:

Check	PowerShell / Bash Command	Expected Result
1. Frontend TypeScript	`cd frontend ; npx tsc --noEmit`	Empty output (0 errors).
2. Backend TypeScript	`cd backend ; npx tsc --noEmit`	Empty output (0 errors).
3. Backend Health API	`curl http://localhost:5000/health`	`{"status":"ok"}`
4. ML Service Health API	`curl http://localhost:8000/health`	`{"status":"healthy"}`
5. MongoDB Connection	`powershell -Command "Test-NetConnection -ComputerName 127.0.0.1 -Port 27017"`	`TcpTestSucceeded : True`
6. Git Working Tree	`git status`	`nothing to commit, working tree clean`

6. Standard Bug Post-Mortem Template

> When documenting a solved bug in team repositories or presentation reports, use this industry-standard 4-line format:
>
> **1. Incident:** What failed and what was observed? (e.g. Schedule showed 0 tasks on dashboard).
> **2. Root Cause:** Why did it happen at code level? (e.g. Time filter was `now → endOfDay` instead of `startOfDay → endOfDay`).
> **3. Surgical Fix:** What line or file was altered? (e.g. Updated query window in `DashboardPage.tsx`).
> **4. Prevention:** How do we ensure it never happens again? (e.g. Added unit test checking schedule queries spanning past morning hours).